

Projet ASD : Structures de Données Hybrides

Seydina Mohamed Diop & Abdou Lahate

19 juillet 2026

1 Introduction

Ce Projet porte sur un systeme de gestion de scolarité . En effet on va concevoir un programme qui va permettre a un utilisateur d enregistrer autant d etudiant qu il souhaite accompagner de leur information personnel et pedagogique .Ainsi une serie de message vont s afficher sur le terminal permettant a l utilisateur d echanger avec la machine .Ce dernier fera des series de calculs liés aux **Structures** et **Fonctions** ci dessous et affichera les informations correspondantes.

2 Modélisation des Structures de Données

Voici les Structures de base que nous avons conçues pour notre **Systeme De Scolarité**.

```
1 // on cree une premier structure de type note
2 typedef struct Note
3 {
4     char matier[20];
5     float valeur;
6     int coef;
7     struct Note *note_suivante; // permet de passer au note
    suivant du meme etudiant
```

2.0.1 Explication de la structure Note

La structure **Note** reçoit et stocke les résultats académiques d'un étudiant pour chaque matière de la filière, ainsi que le coefficient correspondant. Par exemple, pour un étudiant donné, cette structure va enregistrer ses notes au fur et à mesure. Ensuite, grâce au pointeur `struct Note *note_suivante`, la structure se connecte à la note suivante du même étudiant. C'est ce mécanisme

qui forme une liste chaînée de notes propre à chaque étudiant, permettant de stocker un nombre dynamique de modules sans utiliser de tableau statique.

```

1 // on cree une deuxieme structure de type etudiant
2 typedef struct Etudiant
3 {
4     char nom[50];
5     char prenom[50];
6     int numero_dossier; // identifiant unique par etudiant
7     bool eta_inscription; // 1 si l etudiant s'est inscrit et
        0 sinon (initialise lors de la creation)
8     float moyenne_general;
9     Note *modules;
10    struct Etudiant *etudiant_suivant; // permet de passer a
        l etudiant suivant

```

2.0.2 Explication de la structure Etudiant

La structure `Etudiant` regroupe l'ensemble des informations d'un étudiant de la filière, telles que son nom, son prénom, son numéro de dossier (identifiant unique) et son état d'inscription (via un booleen). Le pointeur `modules` est essentielle : il pointe vers la première note de l'étudiant, liant ainsi l'étudiant à ses résultats. Concernant le pointeur `etudiant_suivant` elle pointe sur l'étudiant suivante creent ainsi une liste chaine

```

1 }Etudiant;
2 typedef struct
3 {
4     char nom_filiere[60];
5     int nb_etudiants;
6     Etudiant *premier_du_liste; // point sur le premier de la
        liste des etudiant
7     Etudiant *dernier_du_liste; // point sur le dernier
        etudiant de la liste

```

2.0.3 Explication de la structure Filiere

Cette structure contient le nom d'une filiere donner ainsi que le nombre d'étudiants. Dans la structure `Filiere` on a deux pointeurs : `premier_du_liste` qui pointe sur le premier étudiant de la liste du filière, `dernier_du_liste` quant a lui pointe sur le dernier étudiant. Permettant d inserer des etudiants directemant a la fin de la liste

3 Modélisation des Fonctions

Voici les fonctions que nous avons conçue pour permettre le bon fonctionnement de notre Systeme de scolarité.

```
1 // on va cree une fonction qui permettra de cree un etudiant
  et de l ajouter dans une filiere
2 Etudiant *inscrire_etudiants(Filieres *filiere)
3 {
4     // allocation dynamique d un nouveau etudiant
5     Etudiant *nouveau_etudiant = (Etudiant*) malloc(sizeof(
        Etudiant));
6     if (nouveau_etudiant == NULL)
7     {
8         printf("Erreur: impossible d allouer la memoire");
9         return NULL;
10    }
11    // ajout des information d un etudiant par l utilisateur
12    printf("\n-----Formulaire
        \nd inscription
        -----
        :\n");
13    printf("Donner le nom de l etudiant:\n");
14    scanf("%s", nouveau_etudiant->nom);
15    printf("Donner le prenom de l etudiant:\n");
16    scanf("%s", nouveau_etudiant->prenom);
17    printf("Donner son numero de dossier:\n");
18    scanf("%d", &nouveau_etudiant->numero_dossier);
19
20    nouveau_etudiant->eta_inscription = 1; // on va
        enregistrer l etat d inscription a 1 par default
21    nouveau_etudiant->modules = NULL ; // pas encore de note
22    nouveau_etudiant->etudiant_suivant = NULL; // se sera le
        dernier etudiant
23
24    if (filiere->premier_du_liste == NULL)
25    {
26        // La liste est vide, il est le tout premier et le
        tout dernier
27        filiere->premier_du_liste = nouveau_etudiant;
28        filiere->dernier_du_liste = nouveau_etudiant;
29    } else
30    {
31        filiere->dernier_du_liste->etudiant_suivant =
        nouveau_etudiant; // le pointeur dernier_du_liste
        pointe sure le nouveau etudiant
32        filiere->dernier_du_liste = nouveau_etudiant; // Il
        devient le nouveau dernier de la filiere
33    }
```

```

34
35     filiere->nb_etudiants++;
36     printf("L'etudiant_%s_%s_a_ete_inscrit_avec_succes!\n",
           nouveau_etudiant->prenom, nouveau_etudiant->nom);
37     return nouveau_etudiant;

```

3.0.1 Explication de la Fonctions inscrire_etudiant

Cette fonction nous permet d'ajouter de nouveaux etudiant dans une filiere en enregistrant leur information personnel : prenom, nom, numero de dossier grace a l'allocation dynamique suivant : `Etudiant *nouveau_etudiant = (Etudiant*) malloc(sizeof(Etudiant))` qui alloue une espace memoire dans le memoire RAM. Combiner au boucle suivant.

```

1
2     if (filiere->premier_du_liste == NULL)
3     {
4         // La liste est vide, il est le tout premier et le
           tout dernier
5         filiere->premier_du_liste = nouveau_etudiant;
6         filiere->dernier_du_liste = nouveau_etudiant;
7     } else
8     {
9         filiere->dernier_du_liste->etudiant_suivant =
           nouveau_etudiant; // le pointeur dernier_du_liste
           pointe sure le nouveau etudiant
10        filiere->dernier_du_liste = nouveau_etudiant; // Il
           devient le nouveau dernier de la filiere

```

L'utilisateur pourra ajouter autant d'etudiant qu'il souhaite, le programme aura ainsi le nombre totale d'etudiants.

```

1 }
2 void note_etudiants(Etudiant *etudiant) // une fonction
           permettant d'enregistrer les resultat academique d'un
           etudiant
3 {
4     if (etudiant == NULL) return;
5
6     // On verifie si l'etudiant est bien inscrit
7     if (etudiant->eta_inscription == 0)
8     {
9         printf("Impossible_d'ajouter_une_note_:cet_etudiant_
           n'est_pas_activement_inscrit.\n");
10        return;
11    }
12    int nouveaux_matiere = 2;
13    while (nouveaux_matiere == 2)

```

```

14     {
15         Note *nouveau_note = (Note*)malloc(sizeof(Note)); //
            allocation dynamique
16         if (nouveau_note == NULL)
17         {
18             printf("Erreur : impossible d'allouer la memoire");
19             return;
20         }
21
22
23         // ajout du resultat d un etudiant
24         printf("\n-----Resultats_
            Académique
            -----
            S:\n");
25         printf("Donner la matiere : \n");
26         scanf("%s", nouveau_note->matier);
27         do
28         {
29             printf("Donner la note obtenue : \n");
30             scanf("%f", &nouveau_note->valeur);
31             if (nouveau_note->valeur < 0 || nouveau_note->
                valeur > 20)
32             {
33                 printf("Erreur ! La note doit etre comprise
                    entre 0 et 20.\n");
34             }
35         } while (nouveau_note->valeur < 0 || nouveau_note->
            valeur > 20);
36         do
37         {
38             printf("Donner le coefficient du matiere : \n");
39             scanf("%d", &nouveau_note->coef);
40             if (nouveau_note->coef <= 0 || nouveau_note->coef
                > 10)
41             {
42                 printf("Erreur ! Le coefficient doit etre
                    compris entre 1 et 10.\n");
43             }
44         } while (nouveau_note->coef <= 0 || nouveau_note->
            coef > 10);
45
46         nouveau_note->note_suivante = NULL; // c est la
            premiere et dernier note
47
48         if (etudiant->modules == NULL)
49         {
50             // Cas ou l etudiant n a encore aucune note
51             etudiant->modules = nouveau_note;

```

```

52     }
53     else
54     {
55         // Cas ou l'etudiant a deja des notes, on
           parcourt pour aller a la fin
56         Note *note_restante = etudiant->modules;
57         while (note_restante->note_suivante != NULL)
58         {
59             note_restante = note_restante->note_suivante;
60         }
61         // On ajoute la nouvelle note tout a la fin
62         note_restante->note_suivante = nouveau_note;
63     }
64     printf("Matiere_ajouter_avec_succe:\n");
65     printf("Si_vous_vouler_ajouter_une_nouvelle_matiere_
           tape_2_si_non_0:\n");
66     scanf("%d", &nouveaux_matiere);
67 }

```

3.0.2 Explication de la Fonctions note_etudiant

Comme la fonction precedente celui ci Permet de stocke les resultats scolaire de chaque etudiants enregistrer en amont ,grace a l allocation dynamique suivant `Note *nouveau_note = (Note*)malloc(sizeof(Note))`. Cette fonction va permettre a l utilisateur de donner autant de matiere ainsi que les note et coefficient associer. Pour chacune des etudiants qu il enregistre il peut aussi ne pas ajouter de note pour les etudiants qu il souhaite . Et cela grace au boucle `while (nouveaux_matiere == 2)` qui lui dira de taper 2 s il veut ajouter des matiere et 0 sinon .

```

1 // on va cree une fonction pour parcourir et afficher les
  etudiants de la liste
2 void liste_de_la_classe(Filiere *filiere)
3 {
4     if (filiere == NULL || filiere->premier_du_liste == NULL)
5         // verifie si la filiere est vide ou non
6     {
7         printf("Aucun_etudiant_inscrit_dans_la_filiere.\n");
8         return;
9     }
10    printf("\n-----Liste_des_
           etudiants_de_la_classe
           -----:\n");
11    ;
    Etudiant *actuel_liste = filiere->premier_du_liste;

```

```
12     while (actuel_liste != NULL) // affiche les identifiants
13     {
14         printf("Nom: %s, Prenom: %s, Numero de dossier: %d\n",
15                actuel_liste->nom, actuel_liste->prenom,
16                actuel_liste->numero_dossier);
17         actuel_liste = actuel_liste->etudiant_suivant;
18     }
```

3.0.3 Explication de la Fonctions liste_de_la_classe

Quant a cette fonction il affichera la liste des etudiants que l utilisateur a deja enregistrer .Il donne respectivement leur nom, prenom et numero de dossier.

```

1 // fonction qui va afficher le bulletin d un etudiant
2 void bulletin(Filiere *filiere)
3 {
4     if (filiere == NULL || filiere->premier_du_liste == NULL)
5     {
6         printf("Aucun etudiant inscrit dans la filiere.\n");
7         return;
8     }
9
10    printf("\n-----Bulletin des etudiants de la classe\n");
11
12    float somme_des_moyennes_classe = 0;
13    int etudiants_avec_notes = 0; // prend en compte que les etudiants ayant une moyenne
14
15    Etudiant *actuel_liste_info = filiere->premier_du_liste;
16
17    while (actuel_liste_info != NULL)
18    {
19        // On verifie le statut d inscription de chaque etudiant
20        if (actuel_liste_info->eta_inscription == 1)
21        {
22            printf("Nom: %s, Prenom: %s, Numero de dossier: %d\n", actuel_liste_info->nom,
23                actuel_liste_info->prenom, actuel_liste_info->numero_dossier);
24
25            Note *actuel_liste_bulletin = actuel_liste_info->modules;
26
27            float point_totale = 0;
28            int coef_totale = 0;

```

```

26
27     while (actuel_liste_bulletin != NULL)
28     {
29         printf("Matiere: %s, Note: %f, Coeficient: %d\n",
                actuel_liste_bulletin->matier,
                actuel_liste_bulletin->valeur,
                actuel_liste_bulletin->coef);
30         point_totale += actuel_liste_bulletin->valeur
                        * actuel_liste_bulletin->coef;
31         coef_totale += actuel_liste_bulletin->coef;
32         actuel_liste_bulletin = actuel_liste_bulletin
                                ->note_suivante;
33     }
34
35     if (coef_totale != 0)
36     {
37         actuel_liste_info->moyenne_general =
            point_totale / coef_totale;
38         printf("Moyenne Generale: %.2f\n\n",
            actuel_liste_info->moyenne_general);
39         somme_des_moyennes_classe +=
            actuel_liste_info->moyenne_general;
40         etudiants_avec_notes++;
41     }
42     else
43     {
44         actuel_liste_info->moyenne_general = 0;
45         printf("Aucune note enregistre\n");
46     }
47 }
48 // On passe a l etudiant suivant
49 actuel_liste_info = actuel_liste_info->
    etudiant_suivant;
50 }
51
52 // Affichage de la moyenne gennerale
53 if (etudiants_avec_notes != 0)
54 {
55     float moyenne_general_classe =
        somme_des_moyennes_classe / etudiants_avec_notes;
56     printf("MOYENNE GENERALE DE LA FILIERE: %.2f / 20\n",
        , moyenne_general_classe);
57 }
58 else
59 {
60     printf("Les etudiants non pas de note.\n");
61 }
62 }

```

3.0.4 Explication de la Fonctions bulletin

La fonction `bulletin` permet de calculer les moyenne de chaque etudiant ainsi que la moyenne gennerale de la classe. Elle va aller chercher les module de chaque etudiants c est a dire ces matiere ainsi que les coef et notes associer pour calculer sa moyenne. Si l etudiant n a pas de note la boucle :

```
1         else
2         {
3             actuel_liste_info->moyenne_general = 0;
4             printf("Aucune note enregistre\n");
5         }
```

Va afficher a l ecran que cette etudiant n a pas de note ,cette etudiant ne sera pas prise en compte pour le calcule du moyenne gennerale de la classe . Grace au variable `etudiants_avec_notes` , innicialise a 0 qui n avance que si l etudiant qu on doit calculer la moyenne a une note .De cette facon avec la boucle :

```
1     if (etudiants_avec_notes != 0)
2     {
3         float moyenne_general_classe =
4             somme_des_moyennes_classe / etudiants_avec_notes;
5         printf("MOYENNE GENERALE DE LA FILIERE : %.2f / 20\n",
6             , moyenne_general_classe);
7     }
8     else
9     {
10        printf("Les etudiants non pas de note.\n");
11    }
12 }
```

Cqlcule et affiche la moyenne gennerale exacte de la classe.

```
1 void recherche_et_modification_etudiant(Filieres *filieres)
2 {
3     int votre_choix = 3;
4     int choix;
5     char matier[50];
6     float note;
7     int coef;
8     while (votre_choix == 3)
9     {
10        printf("Voulez vous rechercher ou modifier les info d
11            un autre etudiant si oui taper 3 si non 0:\n");
12        scanf("%d", &votre_choix);
13        if (votre_choix == 0)
14        {
15            break;
16        }
```

```

16     if (filiere == NULL || filiere->premier_du_liste ==
17         NULL)
18     {
19         printf("Aucun étudiant dans la filiere.\n");
20         return;
21     }
22     int id_rechercher;
23     printf("Donner le numero de dossier de l'étudiant
24         rechercher:\n");
25     scanf("%d", &id_rechercher);
26
27     Etudiant *etudiant_actuel = filiere->premier_du_liste
28         ;
29     while (etudiant_actuel != NULL && etudiant_actuel->
30         numero_dossier != id_rechercher)
31     {
32         etudiant_actuel = etudiant_actuel->
33             etudiant_suivant;
34     }
35     if (etudiant_actuel == NULL)
36     {
37         printf("%d ne correspond à aucun étudiant
38             enregistré:\n", id_rechercher);
39     }
40     else
41     {
42         printf("\n
43             -----
44             Etudiant trouvé
45             -----
46             \n");
47         printf("Nom: %s, Prenom: %s, Numero de dossier: %d,
48             Etat d'inscription: %s", etudiant_actuel->
49             nom, etudiant_actuel->prenom, etudiant_actuel
50             ->numero_dossier, (etudiant_actuel->
51             eta_inscription == 1) ? "Inscrit" : "Non
52             inscrit");
53         printf("\n-----Que
54             souhaitez-vous modifier
55             -----
56             \n");
57         printf("[1] Le Nom\n");
58         printf("[2] Le Prenom\n");
59         printf("[3] Le Numero de dossier\n");
60         printf("[4] L'etat d'inscription (1: inscrit, 0:
61             Suspendu)\n");
62         printf("[5] Matiere Note et Coef\n");
63         printf("Votre choix: ");
64         scanf("%d", &choix);

```

```

46     }
47     switch (choix) // cette partie va permettre a l
        utilisateur de modifier les parametre qu il
        souhaite
48     {
49         case 1:
50             printf("Entrez le nouveau nom : \n");
51             scanf("%s", etudiant_actuel->nom);
52             printf("Nom modifie avec succes ! \n");
53             break;
54         case 2:
55             printf("Entrez le nouveau prenom : \n");
56             scanf("%s", etudiant_actuel->prenom);
57             printf("Prenom modifie avec succes ! \n");
58             break;
59         case 3:
60             printf("Entrez le nouveau numero de dossier
                : \n");
61             scanf("%d", &etudiant_actuel->numero_dossier
                );
62             printf("Numero de dossier modifie avec succes
                \n");
63             break;
64         case 4:
65             printf("Entrez le nouveau statut (1 pour
                Inscrit, 0 pour Non inscrit) : \n");
66             scanf("%d", &etudiant_actuel->
                eta_inscription);
67             printf("Statut d'inscription mis a jour ! \n")
                ;
68             break;
69         case 5:
70             if (etudiant_actuel->modules == NULL)
71             {
72                 etudiant_actuel->modules = malloc(sizeof*(
                    etudiant_actuel->modules));
73                 if (etudiant_actuel->modules == NULL)
74                 {
75                     printf("Ereur de l'allocation dynamique
                        ");
76                 }
77             }
78             Note *etudiant_actuel_modules = NULL;
79             Note *actuel = etudiant_actuel->modules;
80             while (actuel->note_suivante != NULL)
81             {
82                 actuel = actuel->note_suivante;

```

3.0.5 Explication de la Fonctions recherche_et_modification_etudiant

Cette fonction va demander a l'utilisateur s'il veut rechercher un étudiant afin de modifier ou non ces informations personnelles ou académiques en lui affichant un message lui permettant de faire deux choix soit taper 3 pour continuer et avoir la possibilité de modifier certaines informations. Soit taper 0 pour arrêter le programme et éventuellement ne rien modifier ou sortir après des modifications.

```
1      etudiant_actuel_modules = actuel;
2
3      printf("Entrez la nouvelle matiere:\n");
4      scanf("%s", matier);
5      if (strcmp(etudiant_actuel_modules->matier,
6      matier) == 0 )
7      {
8          printf("Entrez la nouvelle note:\n");
9          scanf("%f", &etudiant_actuel_modules->valeur);
10         ;
11         printf("Entrez le nouveau coef:\n");
12         scanf("%d", &etudiant_actuel_modules->coef);
13         printf("La matiere ainsi que sa note et son
14         coef ont ete modifier avec succes!\n");
15     }
16     else
17     {
18         etudiant_actuel_modules->note_suivante =
19         malloc(sizeof(Note));
20         if (etudiant_actuel_modules->note_suivante ==
21         NULL)
22         {
23             printf("Erreur d'allocation memoire\n");
24             break;
25         }
26         strcpy(etudiant_actuel_modules->note_suivante
27         ->matier, matier);
28         printf("Donner la note de la nouvelle matier
29         :\n");
30         scanf("%f", &note);
31         etudiant_actuel_modules->note_suivante->
32         valeur = note;
33         printf("Donner le coef de la nouvelle matier
34         :\n");
35         scanf("%d", &coef);
36         etudiant_actuel_modules->note_suivante->coef
37         = coef;
38         etudiant_actuel_modules->note_suivante->
39         note_suivante = NULL;
```

```

30         }
31         break;
32     }
33 }
34 }
35 // fonction pour supprimer un etudiant de la liste
36 void supprimer_etudiant(Filiere *filiere)
37 {
38     int votre_choix = 4;
39     int id_supprimer;
40     while (votre_choix == 4)
41     {
42         printf("Si vous voulez supprimer une autre etudiant_
43             tape 4 sinon 0:\n");
44         scanf("%d", &votre_choix);
45         if (votre_choix == 0)
46         {
47             break;
48         }
49         if (filiere == NULL || filiere->premier_du_liste ==
50             NULL)
51         {
52             printf("Il n'y a aucune etudiant_");
53             return;
54         }
55         printf("Donner le numero de dossier de l etudiant_ que
56             vous souhaitez supprimer de la filiere:\n");
57         scanf("%d", &id_supprimer);
58         Etudiant *actuel_etudiant = filiere->premier_du_liste
59             ;
60         Etudiant *precedent_etudiant = NULL;
61         while (actuel_etudiant != NULL && id_supprimer !=
62             actuel_etudiant->numero_dossier)
63         {
64             precedent_etudiant = actuel_etudiant; // 1
65             etudiant precedent devient 1 actuel etudiant

```

3.0.6 Explication de la Fonctions supprimer_etudiant

De la meme manier que la fonction precedente celui ci lui permetra de supprimer ou non un etudiant de la liste

```

1     }
2     if (actuel_etudiant == NULL)
3     {
4         printf("Etudiant introuvable avec l id: %d\n",
5             id_supprimer);

```

```

5         }
6     else
7     {
8         if (precedent_etudiant == NULL)
9         {
10            // correspond au premier etudiant
11            filiere->premier_du_liste = actuel_etudiant->
                etudiant_suivant;
12            // cas ou la liste contient q un seul etudiant
13            if (filiere->premier_du_liste == NULL)
14            {
15                filiere->dernier_du_liste = NULL;
16            }
17        }
18    }
19    else
20    {
21        // 1 etudiant est au milieu ou a la fin
22        precedent_etudiant->etudiant_suivant =
            actuel_etudiant->etudiant_suivant;
23        if (actuel_etudiant == filiere->
            dernier_du_liste)
24        {
25            filiere->dernier_du_liste =
                precedent_etudiant;
26        }
27    }

```

3.0.7 Explication de la Fonctions maximum

Cette fonction va chercher parmi tout les etudiant celui qui a la plus grande moyenne et a la fin du programme il donne sont nom prenom numero de dossier sa moyenne et le nom de la filiere.

```

1         filiere->nb_etudiants--;
2         printf("L'etudiant avec l'id:%d a ete supprimer\n", id_supprimer);
3     }
4 }
5 }
6 // fonction pour trouver le majorant de la filiere
7 void maximum(Filiere *filiere)
8 {
9     if (filiere == NULL || filiere->premier_du_liste == NULL)
10    {
11        printf("Il n'y a aucune etudiant");
12        return;
13    }

```

```

14     float majorant = 0.0;
15     Etudiant *etudiant_major = filiere->premier_du_liste;
16     Etudiant *le_major = NULL; // Pointeur pour memoriser le
        majorant
17     while (etudiant_major != NULL)
18     {
19         if (etudiant_major->eta_inscription == 1 &&
            etudiant_major->moyenne_general >= 0.0)
20         {
21             if (etudiant_major->moyenne_general > majorant)
22             {
23                 majorant = etudiant_major->moyenne_general;
                // mise a jour du nouveaux majorant
24                 le_major = etudiant_major; // memorise l
                enplacement du majorant actuelle
25             }
26         }
27         etudiant_major = etudiant_major->etudiant_suivant;

```

3.0.8 Explication de la Fonctions minimum

Pareil pour la fonction minimum ,il donne l etudiant ayant la plus faible moyenne de la filiere .

```

1     if (le_major != NULL)
2     {
3         printf("Le_majorant_de_la_filiere:%s,est%s,%s,
            numero_de_dossier:%d,et_a_une_moyenne_generale
            de:%.2f\n",filiere->nom_filiere, le_major->prenom
            , le_major->nom, le_major->numero_dossier,
            le_major->moyenne_general);
4     }
5 }
6 // fonction pour trouver le minorant
7 void minimum(Filiere *filiere)
8 {
9     if (filiere == NULL || filiere->premier_du_liste == NULL)
10    {
11        printf("Il_n_y_a_aucune_etudiant");
12        return;
13    }
14    float minorant = 20;
15    Etudiant *etudiant_minor = filiere->premier_du_liste;
16    Etudiant *le_minor = NULL; // Pointeur pour memoriser le
        minorant
17    while (etudiant_minor != NULL)
18    {
19        if (etudiant_minor->eta_inscription == 1 &&
            etudiant_minor->moyenne_general >= 0.0)

```

```

20     {
21         if (etudiant_minor->moyenne_general < minorant)
22         {
23             minorant = etudiant_minor->moyenne_general;
24             // mise a jour du nouveaux minorant
25             le_minor = etudiant_minor; // memorise l
26             // emplacement du minorant actuelle
27         }
28     }
29     etudiant_minor = etudiant_minor->etudiant_suivant;
30 }
31 if (le_minor != NULL)
32 {
33     printf("L'etudiant ayant la plus faible note de la
34     filiere: %s, est %s, %s, numero de dossier: %d,
35     et a une moyenne generale de: %.2f\n", filiere->
36     nom_filiere, le_minor->prenom, le_minor->nom,
37     le_minor->numero_dossier, le_minor->
38     moyenne_general);
39 }
40 }
41 // tri par ordre de merite (tri par selection)
42 void du_majorant_minorant(Filiere *filiere)
43 {
44     if (filiere == NULL || filiere->premier_du_liste == NULL)
45     {
46         printf("Il n'y a aucune etudiant");
47         return;
48     }
49     Etudiant *i = filiere->premier_du_liste;
50     Etudiant *j;
51     Etudiant *major_minor = NULL; // Pointeur pour memoriser
52     le i
53     // il nous faut des variable et une pointeur pour les
54     echange fineaux
55     char tmp_nom[50];
56     char tmp_prenom[50];
57     int tmp_id;
58     int tmp_etat_inscription;
59     float tmp_moyenne_general;
60     Note *tmp_modules;
61     while (i != NULL)
62     {
63         if (i->eta_inscription == 1)
64         {

```

3.0.9 Explication de la Fonctions du_majorant_minorant

Cette fonction Permet d'afficher par ordre de merite tout les etudiant de la filiere du majorant a l'etudiant ayant la plus faible moyenne grace a la trie par selection :

```
1      {
2          if (etudiant_minor->eta_inscription == 1 &&
              etudiant_minor->moyenne_general >= 0.0)
3          {
4              if (etudiant_minor->moyenne_general < minorant)
5              {
6                  minorant = etudiant_minor->moyenne_general;
6                  // mise a jour du nouveaux minorant
7                  le_minor = etudiant_minor; // memorise l
                  emplacement du minorant actuelle
8              }
9          }
10         etudiant_minor = etudiant_minor->etudiant_suivant;
11     }
12     if (le_minor != NULL)
13     {
14         printf("L'etudiant ayant la plus faible note de la
              filiere:%s,est%s,%s,numero de dossier:%d,
              et une moyenne generale de: %.2f\n",filiere->
              nom_filiere, le_minor->prenom, le_minor->nom,
              le_minor->numero_dossier, le_minor->
              moyenne_general);
15     }
16 }
17 // tri par ordre de merite (tri par selection)
18 void du_majorant_minorant(Filiere *filiere)
19 {
20     if (filiere == NULL || filiere->premier_du_liste == NULL)
21     {
22         printf("Il n'y a aucune etudiant");
23         return;
24     }
25     Etudiant *i = filiere->premier_du_liste;
26     Etudiant *j;
27     Etudiant *major_minor = NULL; // Pointeur pour memoriser
        le i
28     // il nous faut des variable et une pointeur pour les
        echage fineaux
29     char tmp_nom[50];
30     char tmp_prenom[50];
31     int tmp_id;
32     int tmp_etat_inscription;
33     float tmp_moyenne_general;
```

```

34     Note *tmp_modules;
35     while (i != NULL)
36     {
37         if (i->eta_inscription == 1)

1         j = i->etudiant_suivant;
2         while (j != NULL)
3         {
4             if (j->moyenne_general > major_minor->
                    moyenne_general)
5             {
6                 major_minor = j; // j devient
                                temporairement le plus grand
7             }
8             j = j->etudiant_suivant; // j passe a l
                                etudiant suivant
9         }
10        if (major_minor != i)
11        {
12            strcpy(tmp_nom, i->nom);
13            strcpy(tmp_prenom, i->prenom);
14            tmp_id = i->numero_dossier;
15            tmp_etat_inscription = i->eta_inscription;
16            tmp_moyenne_general = i->moyenne_general;
17            tmp_modules = i->modules;
18
19            strcpy(i->nom, major_minor->nom);
20            strcpy(i->prenom, major_minor->prenom);
21            i->numero_dossier = major_minor->
                    numero_dossier;
22            i->eta_inscription = major_minor->
                    eta_inscription;
23            i->moyenne_general = major_minor->
                    moyenne_general;
24            i->modules = major_minor->modules;
25
26            strcpy(major_minor->nom, tmp_nom);
27            strcpy(major_minor->prenom, tmp_prenom);
28            major_minor->numero_dossier = tmp_id;
29            major_minor->eta_inscription =
                    tmp_etat_inscription;
30            major_minor->moyenne_general =
                    tmp_moyenne_general;
31            major_minor->modules = tmp_modules;
32        }
33        i = i->etudiant_suivant;
34    }
35 }
36

```

```

37
38 }
39 // tri par ordre alphabetique (tri a bulle)
40 void orde_alphabet(Filiere *filiere)
41 {
42     if (filiere == NULL || filiere->premier_du_liste == NULL)
43     {
44         printf("Il n'y a aucune etudiant\n");
45         return;
46     }
47
48     Etudiant *i = filiere->premier_du_liste;
49     Etudiant *j;
50
51     char tmp_nom[50];
52     char tmp_prenom[50];
53     int tmp_id;

```

3.0.10 Explication de la Fonctions orde_alphabet

Concernant cette fonction elle utilise une trie a bulle, et elle va permettre l'affichage de la liste par ordre alphabetique :

```

1         strcpy(i->nom, major_minor->nom);
2         strcpy(i->prenom, major_minor->prenom);
3         i->numero_dossier = major_minor->
           numero_dossier;
4         i->eta_inscription = major_minor->
           eta_inscription;
5         i->moyenne_general = major_minor->
           moyenne_general;
6         i->modules = major_minor->modules;
7
8         strcpy(major_minor->nom, tmp_nom);
9         strcpy(major_minor->prenom, tmp_prenom);
10        major_minor->numero_dossier = tmp_id;
11        major_minor->eta_inscription =
           tmp_etat_inscription;
12        major_minor->moyenne_general =
           tmp_moyenne_general;
13        major_minor->modules = tmp_modules;
14    }
15    i = i->etudiant_suivant;
16 }
17 }
18
19
20 }

```

```

21 // tri par ordre alphabetique (tri a bulle)
22 void orde_alphabet(Filiere *filiere)
23 {
24     if (filiere == NULL || filiere->premier_du_liste == NULL)
25     {
26         printf("Il n'y a aucune etudiant\n");
27         return;
28     }
29
30     Etudiant *i = filiere->premier_du_liste;
31     Etudiant *j;
32
33     char tmp_nom[50];
34     char tmp_prenom[50];

```

4 Modélisation du corps de notre projet : (int main)

C est ici que nous faisons les appellees des differentes fonctions pour donner vie a notre programme.

5 Etude de Complexite

Nous allons etudier la complexite de toutes les fonctions dans le meilleur et pire des cas. Soit n : le nombre d'etudiant inscrite, et m : le nombre de module de chaque etudiant.

5.0.1 Complexite des Fonctions inscrire_etudiant

Cette fonction ne contient que des operations elementaires. Donc par sommation des complexites de chaque ligne de code on obtient un $O(1)$.

5.0.2 Complexite des Fonctions note_etudiant

Dans cette fonction nous avons deux boucles de controle de saisie qui ne dependent ni de n ni de m . Mais du nombre d'erreur de l'utilisateur donc ce sont des $O(1)$. Ce pendant nous avons une troisieme boucle imbriquee : `while (note_restante->note_suivante != NULL)`, ce dernier parcourt tout les notes d'un etudiant jusqu'a la fin pour y ajouter la nouvelle note donc la boucle tourne m fois. La boucle principale `while (nouveaux_matiere == 2)` s'exécute aussi m fois car la fin de la boucle depend du nombre de `matiere`.

saisie par l'utilisateur De ce fait nous avons : Dans le meilleur des cas $O(1)$.
Dans le pire des cas $O(m^2)$.

5.0.3 Complexite du Fonctions liste_de_la_classe

Cette fonction d'affichage ne comporte qu'une seule boucle `while (actuel_liste != NULL)`. Cette boucle parcourt tous les étudiants enregistrés dans la filière donc elle tourne n fois, donc nous aurons : Dans le meilleur des cas : $O(1)$.
Dans le pire des cas : $O(n)$.

5.0.4 Complexite du Fonctions bulletin

Dans cette fonction nous avons une boucle imbriquée : `while (actuel_liste_bulletin != NULL)`, cette boucle parcourt l'ensemble des modules de chaque étudiant, (m fois). Une boucle principale : `while (actuel_liste_info != NULL)` qui parcourt l'ensemble des étudiants (n fois). Ainsi nous avons : Dans le meilleur des cas : $O(1)$. Dans le pire des cas : $O(n \times m)$.

5.0.5 Complexite du Fonctions recherche_et_modification_etudiant

La fonction `recherche_et_modification_etudiant` comporte deux boucles imbriquées : `while (etudiant_actuel != NULL && etudiant_actuel->numero_dossier != id_rechercher)` qui s'exécute tant que l'étudiant recherché n'est pas trouvé et la liste n'est pas finie. `while (actuel->note_suivante != NULL)` qui parcourt toutes les notes d'un étudiant. Et une boucle principale `while (votre_choix == 3)` qui ne dépend pas du nombre d'étudiants mais de l'utilisateur donc c est un $O(1)$. Donc on a : Dans le meilleur des cas : $O(1)$. Dans le pire des cas : $O(n) + O(m)$.

5.0.6 Complexite du Fonctions supprimer_etudiant

Dans cette fonction on a une boucle principale `while (votre_choix == 4)` qui ne dépend pas du nombre d'étudiants mais de l'utilisateur donc c est un $O(1)$. Et une boucle interne `while (actuel_etudiant != NULL && id_supprimer != actuel_etudiant->numero_dossier)` qui parcourt les étudiants en comparant leur id. Donc on a : Dans le meilleur des cas : $O(1)$. Dans le pire des cas : $O(n)$.

5.0.7 Complexite du Fonctions maximum

Cette fonction comporte qu'une seule boucle : `while (etudiant_major != NULL)` qui parcourt l'ensemble des étudiants. Donc nous aurons : Dans le

meilleur des cas : $O(n)$. Dans le pire des cas : $O(n)$.

5.0.8 Complexite du Fonctions minimum

Comme la fonction precedante celui ci presente aussi une seule boucle `while (etudiant_minor != NULL)` parcour aussi l ensemble des etudiant. Ainsi : Dans le meilleur des cas : $O(n)$. Dans le pire des cas : $O(n)$.

5.0.9 Complexite du Fonctions du_majorant_minorant

Cette fonction utilise le trie par selection donc elle comporte une boucle principale et une boucle interne. Chaque une de ces boucle parcour les etudiants . Donc on a : Dans le meilleur des cas : $O(n)$. Dans le pire des cas : $O(n^2)$.

5.0.10 Complexite du Fonctions orde_alphabet

Pareil que la fonction precedante celui ci aussi utilise une trie (trie a bulle). De ce fait elle comporte deux boucle une externe et une interne , chaque boucle parcour aussi l ensemble des etudiants. Donc on a : Dans le meilleur des cas : $O(n)$. Dans le pire des cas : $O(n^2)$.

5.0.11 Etude du temps d execution

Nombre d'étudiants (n)	Temps Tri par selection (s)	Temps Tri a bulle (s)
100	0,000000	0,000000
500	0,000000	0,000000
1 000	0,001000	0,011000
1 500	0,004000	0,017000
10 000	0,750000	1,073000
15 000	0,344000	1,577000
50 000	2,977000	24,124001
80 000	8,488000	71,599998
100 000	16,330000	117,851997

TABLE 1 – Comparaison des temps d exécution des tris

5.0.12 Graphique

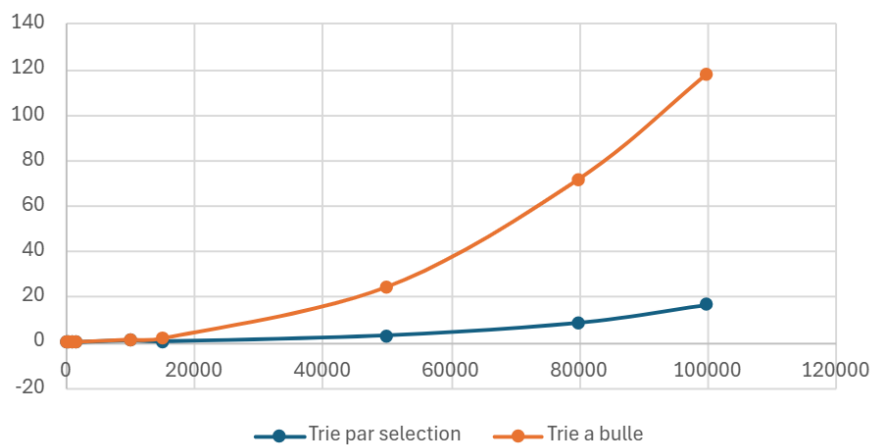


FIGURE 1 – Comparaison du temps d execution entre le tri par selection et le tri a bulle